5회차 · CDC / Iceberg 프로젝트 스터디

현재 상태에서 믿을 수 있는 지표로

dbt · Gold 지표 정의 · 데이터 테스트

Silver의 현재 상태로 정산 지표를 만들고 테스트합니다

흐름을 먼저 알고 설계한다

8회 단계적 진행 로드맵

STEP 1 — 1회차

개념 & 환경 구성

[핵심 학습]

- OT · 왜 CDC+Lakehouse인가
- CDC/Iceberg 내부 구조와 장단점
- 실습 환경 구성 및 공통 예제 흐름 체험

[프로젝트 진행 방식]

공통 예제 (orders·payments·refunds)로 전체 구조 파악

STEP 2 — 2~3회차

수집·적재 흐름 체득

[핵심 학습]

- 2회차: PostgreSQL → Debezium → Kafka 변경 수집
- 3회차: Flink → Iceberg Bronze 적재·지연 측정

[프로젝트 진행 방식]

공통 예제로 c/u/d·체크포인트·커밋을 직접 다룬 뒤, 3회차에 내 개인 프로젝트 도메인·테이블 설계 확정

STEP 3 — 4~6회차

내 프로젝트 본격 구현

[핵심 학습]

- 4회차: Spark SQL Silver 중복 제거·MERGE
- 5회차: dbt Gold 마트·품질 테스트
- 6회차: 과거 데이터 수정·삭제 소급 정정

[프로젝트 진행 방식]

내가 설계한 도메인 테이블과 지표를 파이프라인에 실제 투입·구현

STEP 4 — 7~8회차

성능 튜닝 & 운영 점검

[핵심 학습]

- 7회차: Iceberg 작은 파일 (rewrite_data_files)·파티션·쿼리 튜닝
- 8회차: Airflow 연동·장애 복구 점검·최종 리뷰

[프로젝트 진행 방식]

Iceberg의 단점(Small Files)을 직접 해결하는 전후 성능 비교 벤치마크 완성 (발표 별도)

지난 시간의 결과에서 출발합니다

4회차 Silver 결과

refund-r1 한 행
amount 30000 · is_deleted = true
LSN 300 · offset 12

오늘의 질문

결제 100,000원인 이 주문의
순결제액은 얼마일까요?
(답은 16장에서 확인)

Silver는 현재 상태까지만 알려 줍니다 · 무엇을 더하고 뺄지는 Gold에서 정합니다

정산 담당자는 무엇을 보고 싶을까

먼저 물을 것	공통 예제의 답
누가 쓰나요?	정산 담당자
어떤 판단에 쓰나요?	주문일별로 실제로 남은 결제액 확인
얼마나 빨리 봐야 하나요?	30분쯤 늦은 숫자도 괜찮음 (설계 가정)
어떤 단위로 보나요?	주문 한 건, 그리고 주문일 하루

갱신 주기와 집계 단위는 쓰는 사람의 요구에서 정합니다

오늘 확인할 다섯 가지

확인할 것	오늘의 기준
① 쓰임·지표 정의	정산 담당자·30분 가정 / 성공 결제 – 성공 환불(철회 제외)
② 단위·시간대	주문 1건·주문일 하루 / 서울 기준 날짜
③ 집계 대상·조인	삭제 표시·status로 거름 / 먼저 집계 후 조인
④ 데이터 테스트	키·금액 규칙·기댓값 비교
⑤ 갱신 주기·약속	table 전체 교체 주기 / 설명·소유·열 계약

11장의 Gold 설계 순서와 같은 번호로 진행합니다

Silver에서 Gold로 가는 경로

지난 회차까지의 흐름

오늘: dbt로 Gold 만들기

PostgreSQL
서비스 원천
주문·결제·환불

→

Debezium + Kafka
변경 이벤트
보관

→

Flink
원문 append

→

Iceberg Bronze
변경 이력

→

Iceberg Silver
엔티티 한 행
삭제 표시

→

dbt + Spark
SQL 컴파일
실행 순서
테스트

→

Iceberg Gold
주문별·일별
순결제액

이번 회차의 방식

저장소 dbt 코드와 공통 환경에서 확인한 출력을 함께 읽기
수업 중 새 실행은 하지 않음

실행 역할

dbt: SQL 컴파일·실행 순서·테스트
Spark Thrift: 계산·Iceberg 쓰기

dbt는 SQL을 만들고 순서를 정하며 계산과 쓰기는 Spark가 합니다

dbt는 무엇이고 무엇이 아닌가

dbt가 하는 일

SQL+Jinja를 컴파일
의존 순서대로 실행
테스트·설명을 함께

dbt가 아닌 것

계산 엔진 아님(Spark)
데이터 이동 도구 아님
스케줄러 아님(Airflow)

얻는 것과 치르는 것

얻음: 순서·테스트·설명
치름: Jinja 한 겹,
컴파일 SQL을 따로 확인

대안: Spark SQL 파일을 순서대로 직접 실행 · 모델이 적으면 이것도 충분합니다

프로젝트 파일이 맡는 일

파일	맡는 일	우리 프로젝트
dbt_project.yml	프로젝트 이름·기본 설정	모든 모델 table · iceberg
profiles.yml	어디에 접속하나	Spark Thrift · schema gold
models/sources.yml	원천·모델 설명과 테스트	Silver 3개 · 테스트 5개
models/*.sql	SELECT 하나 = 모델 하나	fct, mart 두 모델
tests/*.sql	규칙을 어긴 행을 찾는 SELECT	settlement_balance

모델 파일에는 CREATE 문이 없고 테이블로 만들지 뷰로 만들지는 설정이 정합니다

source와 ref가 의존성을 만듭니다

source()

원천 테이블을 이름으로
sources.yml에 선언

→

ref()

다른 모델을 참조
실행 순서·계보가 생김

→

컴파일

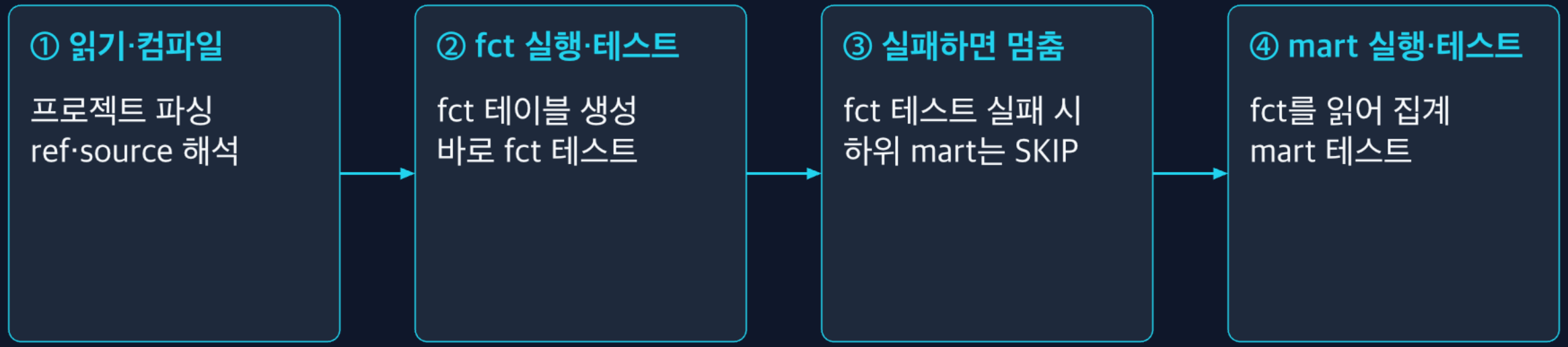
실제 테이블 이름으로
바뀐 SQL을 실행

```
-- fct 모델에 쓴 것
from {{ source('silver','payments') }}
-- dbt compile 결과 (10/31 공통 환경)
from silver.payments
-- mart 모델에 쓴 것
from {{ ref('fct_order_settlement') }}
```

ref·source로 연결해야 dbt가 실행 순서와 계보(lineage)를 압니다

dbt build는 모델마다 실행 후 테스트

dbt run을 다 끝낸 뒤 dbt test를 하는 것과 다릅니다
의존 순서대로 '모델 실행 → 그 모델의 테스트'를 이어 갑니다



fct 테스트가 실패하면 mart는 SKIP되고 이미 바뀐 fct와 시점이 어긋나 알림이 필요합니다

Gold 설계 순서와 아홉 가지 질문

묶음	물을 것	공통 예제
① 쓰임·정의	누가·무슨 판단·얼마나 자주 / 무엇을 더하고 빼나	정산 담당자·30분 / 순결제액
② 단위·시간	한 행은 무엇인가 / 어떤 날짜·시간대	주문 1건, 주문일 하루 / 서울
③ 집계 대상·조인	어떤 행을 세나 / 부풀지 않게 묶나	삭제 표시·success / 먼저 집계
④ 테스트	어떻게 틀릴 수 있고 무엇으로 막나	unique·not_null·규칙·기댓값
⑤ 갱신·관리	얼마나 자주 다시 만드나 / 누가 무엇을 약속	table 30분 / 설명·소유·계약

도메인이 바뀌어도 같은 순서로 묻고 답마다 근거를 함께 적습니다

지표를 먼저 정의합니다

지표	정의	아닌 것
결제액 paid	주문의 성공 결제 합 (삭제 표시 제외)	실패·취소된 결제
환불액 refunded	성공 환불 합 (삭제 표시 제외, 연결된 결제도 같은 조건)	철회된 환불
순결제액 net_paid	paid – refunded, 주문일 기준	회계 매출, 환불일 기준 정산

정의에 '아닌 것'까지 적습니다 · sources.yml: not accounting revenue

한 행은 무엇인가 (집계 단위)

fct_order_settlement

한 행 = 주문 1건
order_id는 한 번만
주문 단위로 검증·추적

mart_daily_settlement

한 행 = 주문일 하루
order_date는 한 번만
fct를 다시 모아 만듦

주문 단위를 거쳐야 이상한 날짜를 주문까지 되짚을 수 있습니다

날짜와 시간대

저장된 값

ordered_at TIMESTAMP
Spark 세션 TZ = UTC
(공통 환경 설정)

변환

UTC 시각을
서울 벽시계 시각으로

주의

세션 TZ가 다르면
결과가 달라짐
내 환경부터 확인

```
to_date(from_utc_timestamp(o.ordered_at, 'Asia/Seoul'))
-- 2026-10-30 15:30 (UTC) → 2026-10-31 00:30 (서울) → 2026-10-31
```

날짜 하나에도 '어느 시간대의 날짜인가'를 정의에 적습니다

어떤 행을 셀까

테이블	조건	왜
orders	not is_deleted	삭제된 주문 제외
payments	not is_deleted · status = 'success'	실패·취소 결제 제외
refunds	not is_deleted · status = 'success'	철회된 환불 제외 (4회차 삭제 표시)
환불에 연결된 결제	결제도 not is_deleted · success	삭제됐거나 success가 아닌 결제의 환불 제외

Silver가 남긴 삭제 표시를 Gold가 거릅니다 · Silver의 '키당 한 행'이 조인의 전제입니다

3장의 답: 철회된 환불의 순결제액

4회차 Silver

결제 100,000원 · success
환불 30,000원 → 철회
is_deleted = true

Gold 계산

paid 100,000
refunded 0 (철회 제외)
net_paid 100,000

공통 검증 스크립트(verify.py)도 이 경우 100,000원을 기댓값으로 확인합니다

조인이 숫자를 부풀리는 경우

먼저 조인하고 합하면

결제 1행 × 환불 2행 → 2행
sum(결제) = 200,000 ✗
sum(환불) = 30,000

먼저 집계하고 조인하면

결제 합 100,000 (주문당 1행)
환불 합 30,000 (주문당 1행)
순결제액 70,000 ✓

1:N 관계에서 한쪽 합계가 행 수만큼 복제됩니다 · 손 계산 예시 (환불 10,000 + 20,000)

fct 읽기 ① 먼저 집계하는 두 CTE

```
-- 결제: 주문별 합
with payments as (
  select order_id, sum(amount) paid
  from {{ source('silver','payments') }}
  where not is_deleted and status='success'
  group by order_id
),
-- 환불: 결제에 붙여 order_id를 찾고 주문별 합
refunds as (
  select p.order_id, sum(r.amount) refunded
  from {{ source('silver','refunds') }} r
  join {{ source('silver','payments') }} p on r.payment_id=p.payment_id
  where not r.is_deleted and not p.is_deleted
    and r.status='success' and p.status='success'
  group by p.order_id
)
```

두 CTE 모두 결과가 '주문당 한 행'입니다 · 저장소 SQL 발췌에 설명 주석만 더했습니다

fct 읽기 ② 주문에 붙이고 mart로

```
select o.order_id,
       to_date(from_utc_timestamp(o.ordered_at,'Asia/Seoul')) order_date,
       coalesce(p.paid,0) paid, coalesce(r.refunded,0) refunded,
       coalesce(p.paid,0)-coalesce(r.refunded,0) net_paid,
       current_timestamp() mart_built_at
from {{ source('silver','orders') }} o
left join payments p on o.order_id=p.order_id
left join refunds  r on o.order_id=r.order_id
where not o.is_deleted
-- mart: from {{ ref('fct_order_settlement') }} group by order_date
```

left join + coalesce: 결제·환불이 없는 주문도 0으로 남습니다

데이터 테스트의 종류

generic 테스트

unique · not_null
accepted_values
relationships
YAML 한 줄로

singular 테스트

규칙을 어긴 행을
반환하는 SELECT 파일
0행이면 통과

unit 테스트

고정 입력으로
모델 로직 확인
우리 조합: 실행 오류

데이터 테스트는 지금 만들어진 테이블의 데이터를 검사합니다

우리 프로젝트의 테스트 여섯 개

테스트	보장하는 것	보장하지 못하는 것
fct order_id unique·not_null	주문당 한 행	금액이 맞는지
fct order_date not_null	날짜가 비지 않음	시간대가 맞는지
mart order_date unique·not_null	하루 한 행	합계가 맞는지
settlement_balance	net = paid − refunded, paid·refunded ≥ 0	부푼 금액, 초과 환불

실측(10/31): dbt test PASS=6 · 통과는 정의한 조건까지만 보장합니다

지금 테스트로 잡을 수 있을까

틀릴 수 있는 방식	지금 6개로?	추가할 테스트 (설계)
조인 fan-out으로 결제액 2배	못 잡음	환불 2건+ 검증 입력 기댓값, 결제 합계 비교
없는 결제를 가리키는 환불	못 잡음	원천 relationships: refunds → payments
예상 밖 status 값	못 잡음	원천 accepted_values: status
환불이 결제보다 큼	못 잡음	singular: refunded > paid

'어떻게 틀릴 수 있나'를 먼저 적고 하나씩 막습니다 · 오른쪽 열은 실행 전 설계입니다

테스트 결과 읽기

명령

dbt test
테이블은 읽기만

이름 규칙

테스트_모델_열
예: unique_fct_..._order_id

실패하면

실패 행 수 확인
컴파일된 SQL을
직접 실행해 원인 행

```
Running with dbt=1.12.5 · Registered adapter: spark=1.9.3
4 of 6 PASS settlement_balance
5 of 6 PASS unique_fct_order_settlement_order_id
Done. PASS=6 WARN=0 ERROR=0 SKIP=0
```

실측(10/31) · 실패하면 target/compiled의 테스트 SQL로 원인 행을 찾습니다

table 모델은 매번 다시 씁니다

① create or replace

dbt-spark가 Iceberg에
테이블을 통째로 교체
(RTAS)

② 새 snapshot

최근 5개 모두 overwrite
parent_id 없음
(10/31 관찰)

③ 쌓이는 이력

fct snapshot 총 902개
(10/31 확인)
전체 교체가 반복됨

읽는 쪽은 교체 전 또는 교체 후 한 상태만 봅니다 · 오래된 snapshot 정리는 7회차

Gold는 얼마나 자주 다시 만들까

선택	신선도	비용·주의
지금: 5분마다 Silver→dbt build 한 작업	최대 5분 + 실행 시간	하루 최대 288번 전체 교체
제안: Silver 5분 / Gold 30분 분리	30분 + 실행 시간 + Silver 지연	하루 48번 · 신선도 요구와 다시 대조
더 빨라야 한다면	분 단위	증분 모델·다른 실행 방식 검토

주기는 4장의 '얼마나 빨리 봐야 하나'에서 정합니다 · 분리는 설계안입니다

증분 모델 예고

table (지금)

매번 전체를 다시 계산
단순하고 결과가 분명
데이터가 크면 비쌈

incremental (6회차)

바뀐 부분만 반영
'바뀐 행'을 찾는 기준 필요
삭제·늦은 정정까지 처리

6회차 질문: 바뀐 것만 반영하면서도 과거 정정과 삭제를 놓치지 않으려면?

협업을 위한 약속 ① 설명·계보·소유

설명 (description)

무엇인지, 무엇이 아닌지
예: not accounting revenue

계보 (lineage)

source → ref로 연결
dbt ls·docs로 확인

소유 (owner)

누가 답하나
groups·meta로 표시

변환 SQL과 그 설명·테스트를 같은 저장소에서 함께 리뷰하고 고칩니다

협업을 위한 약속 ② 모델 계약

약속하는 것

열 이름·타입
빌드 전에 확인

→

우리 조합의 한계

Spark에서 constraints
강제되지 않음
(NOT_ENFORCED)

→

그래서

계약은 열 모양을
데이터 내용은
테스트가 확인

```
models:
  - name: fct_order_settlement
    config: {contract: {enforced: true}}
    columns: [{name: order_id, data_type: string},
              {name: net_paid, data_type: bigint}, ...]
```

설계 예시 · 저장소에 적용·실행하지 않았습니다 (동작은 설치 코드로 확인)

내 테이블로 Gold 설계하기 (약 12분)

묶음	내 도메인으로 적을 것
① 쓰임·정의	누가·무슨 판단·얼마나 자주 / 지표 하나와 '아닌 것'
② 단위·시간	한 행은 무엇 / 어떤 날짜, 어느 시간대
③ 집계 대상·조인	셀 행의 조건 / 1:N에서 부풀지 않는 조인 순서
④ 테스트	틀릴 수 있는 방식 세 가지와 막을 테스트
⑤ 갱신·관리	다시 만드는 주기·방식 / 설명 한 줄·소유자

진행: 예시 2분 → ①② 4분 → ③④⑤ 4분 → 채팅 공유 2분

남길 결과와 다음 회차

오늘 남길 것

지표 정의와 '아닌 것'
집계 단위·셀 행의 조건
조인 순서·테스트 세 개

확인한 사실

dbt build는 실행·테스트를 함께
table은 매번 전체 교체
Spark 계약은 모양까지

6회차

증분 모델
수정·삭제·소급 정정
다시 계산해도 같은 답인가

다음 시간: 바뀐 것만 반영하면서도 과거 정정과 삭제를 놓치지 않는 법